**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# LESSON 033 · C# Login Project · Code a Hash Table

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. A login check uses validUser && validPassword . What is the result when validUser is true and validPassword is false? Explain.

Q2. A D-type flip-flop currently stores Q = 1. D changes to 0 but there is no triggering clock edge. What is Q?

Q3. A queue receives A, B and C in that order. Remove one item, then enqueue D. List the remaining items from front to rear.

Q4. A five-bucket hash table uses h(k) = k MOD 5 with linear probing. Insert keys 7, 12 and 17 in that order into an empty table. State each final index.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

hash table — A table using a hash function to locate data from a key.

hash function — A repeatable mapping from key to bucket index.

bucket — A location in the hash table.

collision — Two keys map to the same bucket.

linear probing — Resolve collisions by checking successive buckets.

key — The value used to look up an entry.

lookup — Find a stored item using its key.

## New learning

The problem: more than one user — A single username comparison is not scalable. We need a way to store and find many usernames.

What is a hash function? — Convert the username to a number, then use % tableSize to choose a bucket index. Our small function is deliberately simple for learning.

What if two names choose one bucket? — That is a collision. Use linear probing: try the next bucket, wrapping to the beginning when necessary.

Lookups must use the same rule — Recalculate the starting bucket from the username, then probe through occupied buckets until the username or an empty bucket is found.

Hash-table lookup versus password hashing — The hash-table function locates an account. It does NOT securely protect a password. Use invented demo passwords only.

MCQ revision — Write four precise MCQs on linked lists, stacks, queues and trees; extend with BST and hash-table MCQs if time allows.

## Worked example

Build a real hash-table index

1. Create the buckets — CODE:
```
const int SIZE = 7;
string?[] usernames = new string?[SIZE];
```

2. Write the index function — CODE:
```
static int Hash(string key, int size)
{
```

```
int total = 0;
foreach (char letter in key)
total += (int)letter;
return total % size;
}
```

3. Explain the limitation — CODE:

```
// Hash("ab", 7) == Hash("ba", 7)
// Same total -> collision.
// This is an educational hash function,
// NOT a secure password hash.
```

## We Do · Practise with guidance

Insert with linear probing

What happens when a bucket is already occupied?

Explain your solution first

```
int index = Hash(username, SIZE);
while (usernames[index] != null)
{
    index = (index + 1) % SIZE;
}
usernames[index] = username;
```

Search using the same probing sequence

How can we find a name after a collision?

```
int index = Hash(username, SIZE);
int checkedBuckets = 0;
while (checkedBuckets < SIZE && usernames[index] != null)
{
    if (usernames[index] == username) break;
    index = (index + 1) % SIZE;
    checkedBuckets++;
}
```

Predict a collision

With table size 7, why do 'ab' and 'ba' choose the same initial index?

Both strings contain the same character codes, so the total is identical. Linear probing moves the second username to the next free bucket.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

## Question 1

CODING 1 · INSERT
Create a seven-bucket string array, implement Hash(), and insert at least four test usernames. Print each chosen bucket index.

## Question 2

CODING 2 · COLLISION
Register 'ab' and 'ba'. Show that they collide and use linear probing to store both in different buckets.

## Question 3

CODING 3 · LOOKUP
Write a search that follows the same probing sequence. Return found/not found, including for a username never inserted.

## Question 4

SYNOPTIC MCQ BANK
Write MCQs on linked lists, directed/undirected graphs, stacks, queues, trees, binary search trees and hash tables. Make a six-question draft; each needs four options and a written explanation.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# Exit Ticket · Answer before checking

A hash-table function based on adding character codes is suitable for protecting real passwords.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

# Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. False. AND ( && ) is true only when both conditions are true.

Q2. Q remains 1. The input is sampled only when the triggering clock edge occurs.

Q3. B, C, D. A leaves first because a queue is FIFO.

Q4. 7 hashes to 2 and is stored at 2. 12 also hashes to 2, so probes to 3. 17 also hashes to 2; 2 and 3 are occupied, so it goes to 4.

### You Do pages · Independent questions

Q1. CODING 1 · INSERT
string?[] usernames = new string?[7]; static int Hash(string key, int size) { int total = 0; foreach (char c in key) total += c; return total % size; }. Insert only if the username is not already present and a bucket is available.

Q2. CODING 2 · COLLISION
The ASCII/Unicode scalar totals for 'ab' and 'ba' are equal, so both yield the same index. For table size 7, (97 + 98) % 7 = 6. If 'ab' uses bucket 6, 'ba' wraps to bucket 0 using (6 + 1) % 7.

Q3. CODING 3 · LOOKUP
Recalculate the starting hash and probe, stopping when a matching username is found, a null bucket is reached, or seven positions have been checked. A full cycle must terminate. Never assume each username lives at the original bucket.

Q4. SYNOPTIC MCQ BANK
Question-bank checklist: linked-list follows pointers; directed graph edges have direction; stack LIFO; queue FIFO; tree is acyclic hierarchy; BST orders smaller left and larger right; hash table uses keys, hashing and collision resolution. Explanations are required.

### Exit Ticket · Model answer

False. A simple character-code sum is not a secure password hash. It is only suitable for demonstrating bucket lookup.

# Try the question again

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**